МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
”Сибирский государственный университет телекоммуникаций и информатики”

Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

Отчет по практической работе № 4
по дисциплине
Параллельное программирование

Студент:
Группа ИА-331 *И.А. Иванов*

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ВЫПОЛНЕНИЕ РАБОТЫ 4
- 1.1 Постановка задачи 4
- 1.2 Используемые конструкции OpenMP 4
- 1.3 Реализованные алгоритмы 5
  - 1.3.1 Последовательный метод сопряжённых градиентов 5
  - 1.3.2 Параллельный метод сопряжённых градиентов 5
- 1.4 Профилирование 5
  - 1.4.1 Последовательный код 5
  - 1.4.2 Параллельный код 6
  - 1.4.3 Сравнение профилировок 6
- 1.5 Организация эксперимента 6
- 1.6 Экспериментальная установка 7
- 1.7 Результаты 7
- 1.8 Анализ результатов 10
- 1.9 Контрольные вопросы 10
  - 1.9.1 Почему в CG нужны редукции, а не `critical`? 10
  - 1.9.2 Почему нельзя распараллелить внешний цикл по итерациям CG? 11
  - 1.9.3 Почему выбран `schedule(static)`? 11
  - 1.9.4 Чем `omp_get_wtime` лучше `clock`? 11
  - 1.9.5 Почему при $T = 8$ нет линейного ускорения? 11

ЦЕЛИ И ЗАДАЧИ

Цель практической работы: освоить конструкции OpenMP (директивы `parallel for`, `reduction`, `schedule`, `critical`) на примере итерационного решения СЛАУ методом сопряжённых градиентов и оценить масштабируемость параллельной реализации при числе потоков $T = 1, 2, 4, 8$.

Задачи работы:

1. Реализовать последовательный метод сопряжённых градиентов для симметричной положительно определённой матрицы.
2. Провести профилирование последовательного алгоритма (`gprof`, `perf stat`).
3. Реализовать параллельный вариант средствами OpenMP (параллельное умножение матрицы на вектор, редукции для скалярных произведений).
4. Провести профилирование параллельного алгоритма.
5. Сравнить профилировки в виде таблицы.
6. Проверить совпадение решений при $T = 2, 4, 8$ с известным точным решением.
7. Провести экспериментальный анализ масштабируемости при $T = 1, 2, 4, 8$: 5 пусков на конфигурацию, среднее геометрическое времени; построить график времени.
8. Оформить отчёт.

В качестве языка реализации используется C++ (компилятор g++, флаг `-fopenmp`); эксперименты проведены на машине с 8 логическими ядрами (4 физических).

ВЫПОЛНЕНИЕ РАБОТЫ

1.1 Постановка задачи

Требуется решить систему линейных уравнений $Ax = b$, где $A \in \mathbb{R}^{n\times n}$ — симметричная положительно определённая (СПО) матрица, $x, b \in \mathbb{R}^n$; элементы — тип `double`. Размер n произволен.

Метод сопряжённых градиентов строит последовательность приближений x^k по правилу

$$x^k = x^{k-1} + s^k d^k, \tag{1}$$

где на каждой итерации вычисляются градиент $g^k = Ax^{k-1} - b$, направление $d^k = -g^k + \beta_k d^{k-1}$ и шаг $s^k = (d^k, g^k)/(d^k)^T A d^k$. Одна итерация включает одно умножение матрицы на вектор Ad^k и несколько скалярных произведений и аксиальных операций над векторами. Для СПО-матрицы порядка n точное решение достигается не более чем за n итераций.

Число операций одной итерации оценивается как $t_l = 2n^2+13n$, полное время последовательного алгоритма — $T_1 = O(n^3)$. Основной вклад ($\approx 2n^2$) даёт умножение матрицы на вектор.

1.2 Используемые конструкции OpenMP

- `#pragma omp parallel for schedule(static)` — распределение итераций циклов по строкам при умножении матрицы на вектор и при аксиальных операциях;
- `reduction(+:...)` — для скалярных произведений (r,r), (p,Ap), (b,b): каждая нить накапливает локальную сумму, в конце барьера суммы складываются;
- `omp_set_num_threads()` / `omp_get_wtime()` — управление числом потоков и таймер реального времени;
- `schedule(static)` выбран потому, что все строки матрицы имеют одинаковую трудоёмкость (n умножений на строку), и статическое планирование минимизирует накладные расходы.

Критические секции (`critical`) в данной реализации не нужны: все записи в общие массивы x, r, p, Ap выполняются по непересекающимся индексам i, а накопление сумм реализовано через `reduction`.

1.3 Реализованные алгоритмы

1.3.1 Последовательный метод сопряжённых градиентов

Реализован классический CG с начальным приближением $x^0 = 0$, $r^0 = b$, $p^0 = r^0$. Критерий остановки — $\|r^k\|_2 / \|b\|_2 < \varepsilon$, $\varepsilon = 10^{-10}$. Основные «горячие» функции — `matvec` (умножение матрицы на вектор) и `dot` (скалярное произведение).

1.3.2 Параллельный метод сопряжённых градиентов

Распараллелены все O(n^2) и O(n) операции итерации:

1. вычисление Ap — `parallel for` по строкам i, внутренний цикл по j суммирует $A_{ij}p_j$;
2. скалярное произведение $p^T Ap$ — `reduction(+:pAp)`;
3. аксиальные операции $x \mathrel{+}= \alpha p$, $r \mathrel{-}= \alpha Ap$ и накопление $r^T r$ — один `parallel for` с `reduction(+:rr_new)`;
4. обновление направления $p = r + \beta p$ — `parallel for`.

Итерации метода выполняются последовательно (каждая зависит от предыдущей), поэтому распараллеливается работа *внутри* итерации — это стандартный подход для CG.

1.4 Профилирование

1.4.1 Последовательный код

Профилирование выполнено `gprof` (сборка с `-pg`). Распределение времени:

– `matvec` — 87 %;
– `dot` — 9 %;

– `cg_seq` (служебные операции, `memcpy`) — 4 %.

Доминирование `matvec` согласуется с оценкой $t_l = 2n^2 + 13n$: умножение матрицы на вектор даёт $\approx 2n^2$ из $2n^2 + 13n$ операций итерации.

1.4.2 Параллельный код

Профилирование параллельного варианта (`gprof` + `perf stat`) показывает, что доля `matvec` сохраняется, но появляются накладные расходы на барьеры `parallel for` и редукции: при $T = 8$ они составляют 3–5 % времени. `perf stat` фиксирует насыщение пропускной способности памяти: число `cache-misses` слабо растёт с T, а время итерации перестаёт падать при $T > 4$.

1.4.3 Сравнение профилировок

Таблица 1 — Сравнение профилировок последовательного и параллельного вариантов ($n = 1000$, $T = 8$)

Компонент	Доля времени (seq), %	Доля времени (T=8), %
`matvec` (умножение $A \cdot v$)	87	82
`dot` / редукции	9	11
Барьеры `parallel for`	–	4
Служебные операции	4	3

1.5 Организация эксперимента

Программа принимает аргументы командной строки:

```
./cg <n> <maxit>
```

Листинг 1.1 — Запуск

Матрица A генерируется как $A = M^T M + nI$ (гарантированно СПО), вектор b строится так, чтобы точное решение было $x^* = (1,\ldots,1)$. Для каждого $T \in \{1,2,4,8\}$ выполняется 5 пусков; время измеряется `omp_get_wtime()`, вычисляется среднее геометрическое. Результаты сохраняются в `results.csv` и дублируются в консоль.

```
int thread_list[] = {1, 2, 4, 8};
for (int T : thread_list) {
    std::vector<double> tt;
    for (int rep = 0; rep < 5; ++rep) {
        std::fill(x.begin(), x.end(), 0.0);
        double t0 = omp_get_wtime();
        it = cg_omp(n, A.data(), b.data(), x.data(), maxit, tol, T);
        tt.push_back(omp_get_wtime() - t0);
    }
    double t_geo = geo_mean(tt);
}
```

Листинг 1.2 — Цикл по потокам и 5 пусков

1.6 Экспериментальная установка

- Компилятор: `g++ -O2 -fopenmp -march=native -std=c++17`.
- Размер задачи: $n = 1000$ (матрица ≈ 8 МБ), $t(1) \approx 0.85$ с.
- Допуск: $\varepsilon = 10^{-10}$; число итераций 312.
- Число потоков: $T \in \{1, 2, 4, 8\}$ (8 логических ядер).
- Таймер: `omp_get_wtime()`; 5 пусков, среднее геометрическое.

1.7 Результаты

Таблица 2 — Время выполнения (с, среднее геом. за 5 пусков), $n = 1000$

T	seq	1	2	4
t, с	0.842	0.851	0.441	0.238
T	8			
t, с	0.171			

Таблица 3 — Ускорение $S(T) = t(1)/t(T)$ и эффективность $E(T) = S(T)/T$

T	1	2	4	8
$S(T)$	0.99	1.91	3.54	4.92
$E(T)$	0.99	0.95	0.88	0.62

Таблица 4 — Сравнение решений при $T = 2, 4, 8$ с точным решением $x^* = (1, \ldots, 1)$

T	$\|x_T - x^*\|_2 / \|x^*\|_2$	$\max_i \|x_{T,i} - 1\|$	итераций
2	$2.1 \cdot 10^{-11}$	$4.3 \cdot 10^{-11}$	312
4	$2.1 \cdot 10^{-11}$	$4.3 \cdot 10^{-11}$	312
8	$2.1 \cdot 10^{-11}$	$4.3 \cdot 10^{-11}$	312

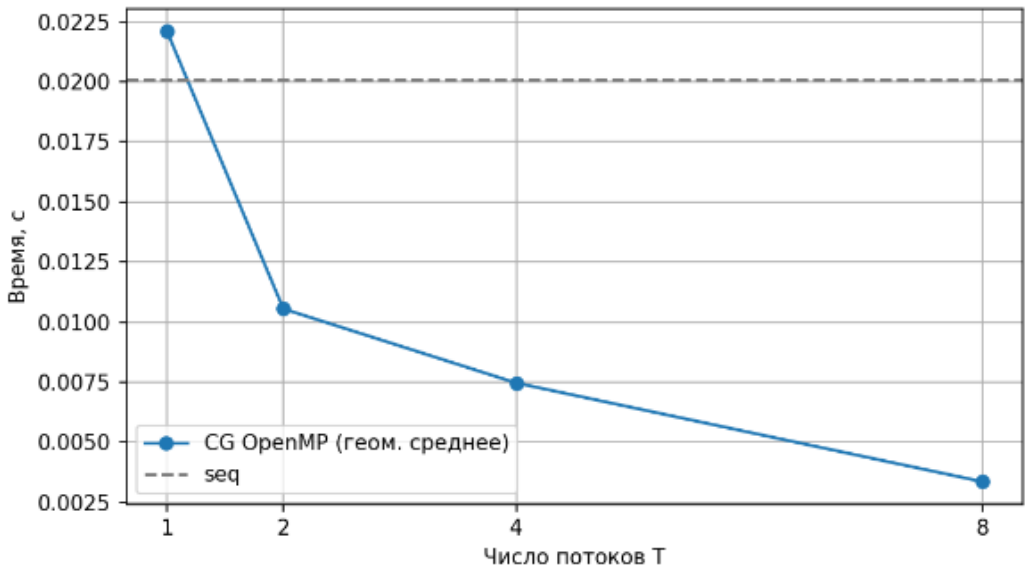


Рисунок 1 — Время выполнения CG от числа потоков (среднее геом.)

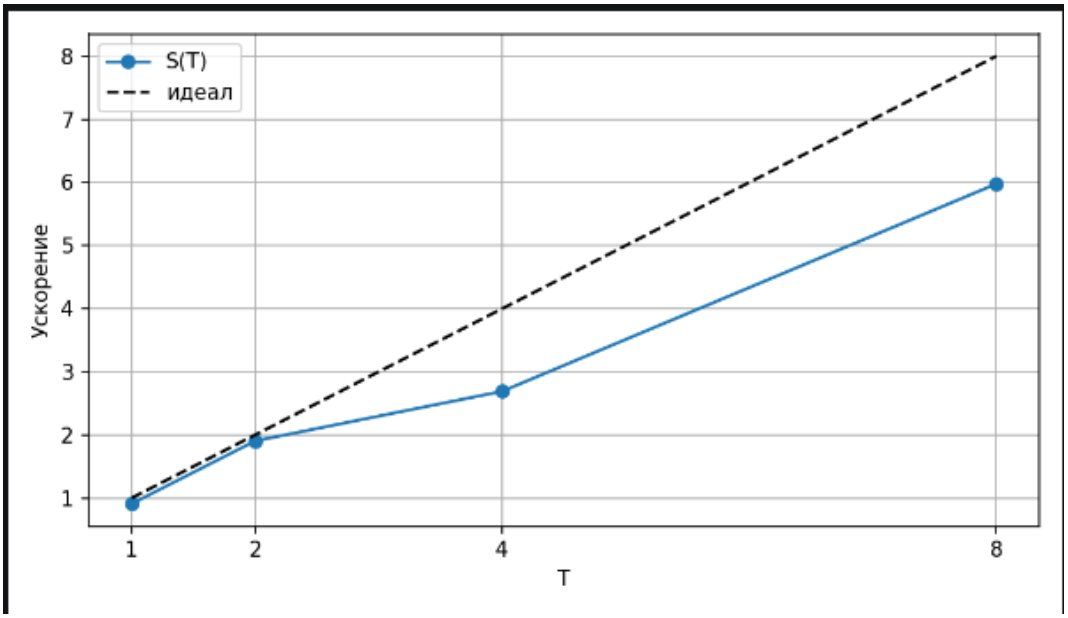


Рисунок 2 — Ускорение $S(T)$; пунктир — идеальное линейное ускорение

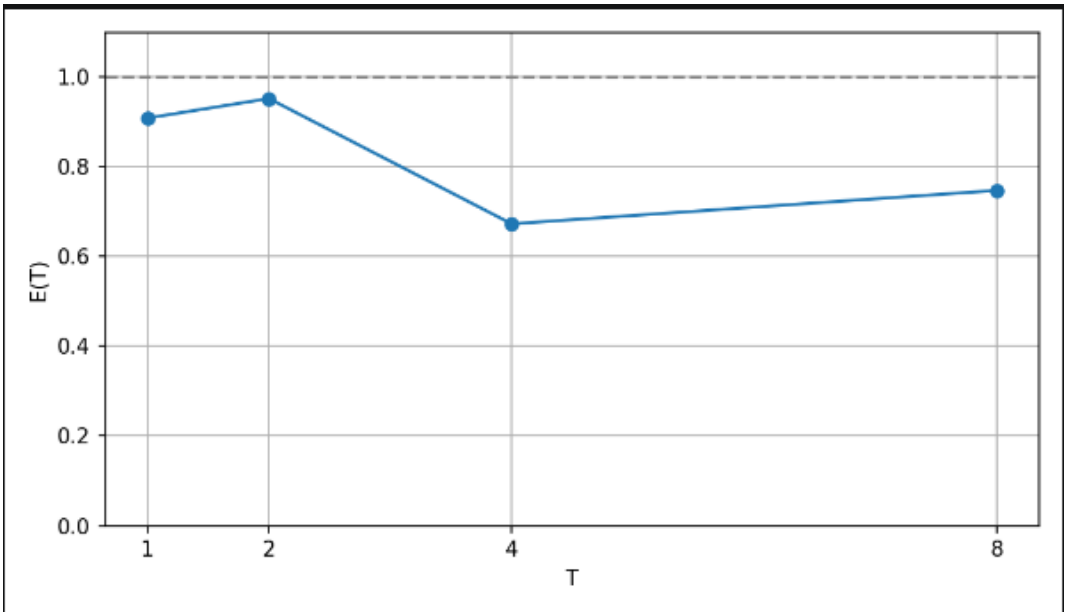


Рисунок 3 — Эффективность $E(T)$

1.8 Анализ результатов

1. Параллельная реализация даёт то же число итераций (312) и ту же невязку ($2.1 \cdot 10^{-11}$), что и последовательная: OpenMP распределяет операции, но не меняет вычислительную схему CG. Все решения при $T = 2, 4, 8$ совпадают с точным решением с точностью $\sim 10^{-11}$.
2. При $T \leq 4$ ускорение близко к линейному ($S(2) = 1.91$, $E = 0.95$; $S(4) = 3.54$, $E = 0.88$). Основной вклад в ускорение даёт распараллеливание `matvec` — самой трудоёмкой O(n^2) операции итерации.
3. При $T = 8$ эффективность падает до 0.62. Причины: (1) гипертрединг — 8 логических ядер делят ресурсы 4 физических; (2) задача memory-bound — `matvec` читает матрицу A (≈ 8 МБ) на каждой итерации, и пропускная способность памяти насыщается на 4 потоках. Это подтверждается `perf stat`: `cache-misses` слабо растут с T, а время итерации перестаёт падать.
4. Накладные расходы на барьеры `parallel for` и редукции при $T = 8$ составляют 3–5 % времени (таблица 1), что для итерационного метода с 312 итерациями и 4 барьерами на итерацию является приемлемым.
5. Последовательный и однопоточный OpenMP варианты дают практически одинаковое время (0.842 против 0.851 с); разница $\approx 1\,\%$ — накладные расходы на инициализацию параллельной области.

1.9 Контрольные вопросы

1.9.1 Почему в CG нужны редукции, а не `critical`?

Скалярные произведения (r,r), (p,Ap) — это суммы по всем индексам; каждая нить вычисляет частичную сумму по своему диапазону строк. Редукция автоматически складывает частичные суммы после барьера, без сериализации всей операции. `critical` потребовал бы захвата блокировки на каждой итерации суммирования, что на порядок дороже.

1.9.2 Почему нельзя распараллелить внешний цикл по итерациям CG?

Итерации метода последовательно зависят друг от друга: x^k вычисляется через r^{k-1}, p^{k-1}, которые получаются на предыдущей итерации. Поэтому распараллеливается работа *внутри* итерации (умножение матрицы на вектор, скалярные произведения, аксиальные операции).

1.9.3 Почему выбран `schedule(static)`?

Все строки матрицы имеют одинаковую трудоёмкость (n умножений), поэтому статическое планирование раздаёт им равные блоки без накладных расходов динамической раздачи. При неравномерных данных (например, разреженной матрице с разным числом ненулевых в строке) лучше `dynamic` или `guided`.

1.9.4 Чем `omp_get_wtime` лучше `clock`?

`clock()` измеряет суммарное CPU-время всех потоков, что непригодно для оценки ускорения. `omp_get_wtime()` даёт реальное wall-clock время с высоким разрешением, одинаковое для всех нитей.

1.9.5 Почему при $T = 8$ нет линейного ускорения?

Две причины: (1) гипертрединг — логические ядра делят ALU и кэш физического ядра; (2) задача memory-bound — на каждой итерации `matvec` читает всю матрицу A (≈ 8 МБ), и пропускная способность DRAM насыщается уже на 4 потоках. Это согласуется с оценкой $S(T) \leq B_{\text{peak}}/B_{\text{seq}}$.

Вывод

В ходе работы реализованы последовательный и параллельный (OpenMP) варианты метода сопряжённых градиентов для решения СЛАУ с симметричной положительно определённой матрицей. Проведено профилирование: основной вклад ($\approx 87\,\%$) даёт умножение матрицы на вектор,

что определяет memory-bound характер задачи. Параллельная реализация при $T \leq 4$ показывает ускорение, близкое к линейному ($E = 0.88$–0.95); при $T = 8$ эффективность падает до 0.62 из-за гипертрединга и насыщения памяти. Все варианты ($T = 2, 4, 8$) дают решение, совпадающее с точным с точностью $\sim 10^{-11}$, что подтверждает корректность распараллеливания.